

Modern Trends in HPC : PETSc

Gaurav Gokhale

School of Computation, Information and Technology

Technical University of Munich

Munich, Germany

gaurav.gokhale@tum.de

Abstract—This seminar paper presents a study of the PETSc (Portable, Extensible Toolkit for Scientific Computation) library. Key features like Linear System Solvers, Nonlinear Solvers, Grid Management, Time Integration, Profiling, and GPU support are discussed. Furthermore, two case studies are presented. Firstly, a nonlinear driven cavity problem. It demonstrates the use of the SNES (Nonlinear Equation Solver) interface to solve a nonlinear problem. The impact of different preconditioning techniques on convergence is explored. Also, a comparison is made for different architectures like CPU and GPU. The second case study consists of experiments with different Krylov Subspace solvers and preconditioners for an existing code framework of the course *Turbulent Flow on HPC Systems* NS-EOF offered at the Technical University of Munich.

I. INTRODUCTION

Modern trends in High-Performance Computing (HPC) have greatly contributed to the development of powerful libraries in scientific computing. They have pushed for advancements in computational capabilities, data handling, and parallel processing, leading to the creation of libraries that can efficiently handle complex scientific computations. These libraries offer scalable parallel algorithms, data structures, and execution models.

PETSc [1] (Portable Extensible Toolkit for Scientific Computation) is one such popular software library for the scalable solution of partial differential equations and linear algebra problems. With a range of solvers and preconditioners for linear and nonlinear systems, optimization issues, grid management, and more, it offers a flexible and adaptable platform for the creation of scientific applications. PETSc is utilized in a range of scientific domains, including physics, engineering, and biology. It is designed to operate effectively on parallel and distributed computing platforms, including CPUs and GPUs. The library has been developed in C and C++ and is open-source. Overall, PETSc is a valuable tool for researchers and developers in a wide range of disciplines.

Alternatives to PETSc also exist, like Ginkgo and Trilinos. They both are open-source software projects for scientific computing. They come with a collection of high-performance linear algebra solvers just like PETSc.

Both are implemented in C++ and have a modular design. They're also highly scalable and support multi-core CPUs, GPUs and other accelerators. [2]

II. THEORETICAL BACKGROUND

Before we delve into the details of how PETSc works in next section, we discuss some fundamental concepts and algorithms which form the basis of implementation inside PETSc.

A. Linear System of Equations

Here's a simple linear system in its Matrix form:

$$\underset{\text{matrix}}{A} \underset{\text{solution}}{u} = \underset{\text{RHS}}{b} \quad (1)$$

The system could be solved by inverting the matrix A as:

$$u = A^{-1}b \quad (2)$$

While this looks simple enough for such a small system, things are not so straightforward with a large linear system prominently due to following reasons [6]:

- There are limitations to the precision with which floating point numbers are represented on computers. On a computer with machine precision ϵ , the solution of the system can be found only upto $O(\kappa(A)\epsilon)$ accuracy, where κ is the condition number of the matrix. Thus, systems having high condition numbers would result in inaccurate solutions
- Inverting a matrix is an expensive operation. Direct solving methods like Gauss-Elimination with LU Decomposition take $O(n^3)$ operations
- The inverse or the LU factorization of a matrix can be dense even if it was originally sparse. Thus inverting those would require more storage, which for bigger problems might not even fit in memory

These problems lead us to iterative methods to solve the system of equations. The basic idea behind iterative methods is to start with an initial guess and improve towards(hopefully) the solution. However, the convergence of such iterative methods is not always guaranteed and depends on the spectral properties of the matrix.

This is why preconditioning is often done to obtain a system with better convergence properties. Preconditioning involves an invertible matrix M and solving a new linear system:

$$(M^{-1}A)u = (M^{-1})b \quad (3a)$$

$$(AM^{-1})(Mu) = b \quad (3b)$$

These systems are called *left* 3a and *right* 3b preconditioned systems of the original system $Au = b$. Such preconditioning is effective only when M^{-1} is easy to apply and approximately inverts A . Jacobi, Gauss-Seidel, SOR etc., are some of the simple iteration methods which arise for different choices of M .

Another class of more powerful iterative methods are the Krylov Subspace methods. Krylov subspaces are generated by applying the given matrix A to a starting vector multiple times. [6]. Methods like Conjugate Gradient(CG), Generalized Minimum Residuals(GMRES), Bi-Conjugate Gradient(BiCG) etc., belong to this category. While discussing the exact working of each method is beyond the scope of this report, it is worthwhile to note that no method is perfect for all problems. The selection of a method often depends on the properties of the system in question and involves some experimentation, which is facilitated by PETSc.

B. Non-Linear System of Equations

A general non-linear system reads as follows:

$$F(x) = b \quad (4)$$

Contrary to the linear systems, only iterative methods exist for solving a nonlinear system. Similar to its linear counterpart, a nonlinear Richardson iterative method is given by:

$$x_{k+1} = x_k + \lambda(b - F(x_k)) \quad (5)$$

Another method is the Newton's method:

$$F(x_k + s) = F(x_k) + J_F(x_k)s + O(\|s\|) \quad (6a)$$

$$x_{k+1} = x_k + s \quad (6b)$$

The equation 6a is obtained by linearizing $F(x)$ around its root by multi-dimensional Taylor's expansion. Here s is the step-size which is taken towards updating the solution and J_F is the Jacobian of the function F . It is given by:

$$J_F = \begin{bmatrix} \frac{\partial F_0}{\partial x_0} & \cdots & \frac{\partial F_0}{\partial x_{N-1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_{N-1}}{\partial x_0} & \cdots & \frac{\partial F_{N-1}}{\partial x_{N-1}} \end{bmatrix}$$

It is often overkill to solve the system $J_F(x_k)s = F(x_k)$ exactly to obtain the step size s . In practice, this system is solved approximately to obtain the step size as follows:

$$\|F'(x_k)s + F(x_k)\| \leq \eta \|F(x_k)\| \quad (7)$$

Such methods are called as Inexact Newton methods. If Krylov subspace solvers are used to solve 7, they are called Inexact Newton Krylov methods. The approximate solution is obtained using the Krylov subspace method as the "inner" linear iterative solver. The parameter η is a scalar between [0,1] [6] that determines the accuracy with which the Jacobian system needs to be solved.

Thus, each Newton's iteration requires solving a linear system using the methods mentioned in the previous section and then an update to the current solution 6b. Also, Newton's method is known to display quadratic convergence if the guess is sufficiently close to the root of the system [6]

Sometimes, the nonlinear solvers are combined or preconditioned with another nonlinear solver. While the finer details of such composed nonlinear solvers are beyond the scope of this paper, their usage is demonstrated in Case Study 1 of a nonlinear lid-driven cavity. Conceptually, the nonlinear preconditioner solver eliminates some solution components that would have been difficult to solve using just the outer solver. For more details [5] and [9] could be referred.

C. Solving Transient PDEs

This section discusses the general workflow for discretization of transient PDEs. Consider a nonlinear heat equation with a source term on a rectangular grid:

$$\frac{\partial u}{\partial t} = \alpha * \Delta u + f(u) \quad (8)$$

where $u(x,t)$ is unknown function for temperature distribution,

α is the thermal diffusivity,

T is the temperature,

$f(u)$ is the nonlinear source term

To discretize this equation, we can use the finite difference scheme. Let the temperature distribution at a grid point (i,j) at the k^{th} timestep be given as $u_{i,j}^k$. The FD discretization of the laplacian would look as :

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{i+1,j}^k - 2u_{i,j}^k + u_{i-1,j}^k}{\Delta x^2} \quad (9a)$$

$$\frac{\partial^2 u}{\partial y^2} = \frac{u_{i,j+1}^k - 2u_{i,j}^k + u_{i,j-1}^k}{\Delta y^2} \quad (9b)$$

Similarly, we can discretize the time derivative, using one of the time integrators, for instance explicit euler.

$$\frac{\partial u}{\partial t} = \frac{u_{i,j}^{k+1} - u_{i,j}^k}{\Delta T} \quad (10)$$

Having discretized the derivative terms, we can now rearrange to obtain a system of nonlinear equations in the form $F(u^{k+1}) = 0$ where F represents the discretized PDE equation. We can then solve this nonlinear system

using the methods mentioned in the previous section. It is important to note here, that each timestep involves solving a nonlinear system, which would itself involve solving a linear system iteratively until convergence. PETSc provides functionalities that assist with each step of solving such PDEs, like discretization, time integrators, linear/nonlinear solvers, preconditioner etc. These features are discussed briefly in the next section.

III. OVERVIEW OF PETSc

The main philosophy of PETSc is to facilitate experimentation. It is imperative to consider the underlying physics, algorithms and architectures to find an optimal solver. Keeping these in mind, PETSc allows its user to set many solver configurations at runtime without the need for recompilation. Moreover, the data layout, choice of architecture(CPU vs GPU), computational kernels etc., can be accessed via the runtime options database. [8]

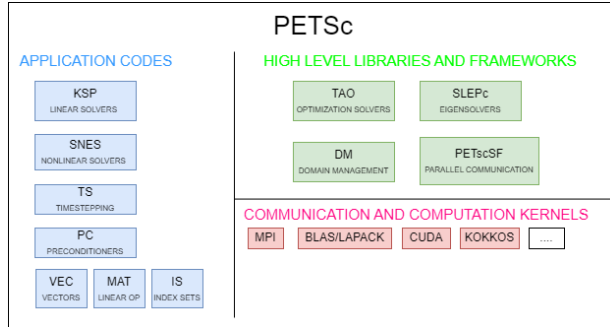


Fig. 1: Structure of PETSc [9]

As can be seen in figure 1, the workhorse of PETSc are the application codes, namely the solvers like *KSP*, *SNES*, *TS*, *PC* and data structures like *VEC*, *MAT* and *IS*. Few of them are discussed below:

A. *KSP Solvers*

The Krylov subspace solvers are a part of the PETSc class *KSP*, and its type can be selected at runtime using `-ksp_type`, and the preconditioner *PC* can be chosen using `-pc_type`. To use the *KSP* solvers, the user has to assemble the system matrix and right-hand side vectors of the system inside PETSc objects *Mat* and *Vec*. Details about the selected *KSP*, *PC* and various other things like type of matrix storage, convergence tolerances, number of non-zeros etc. can be obtained by setting the `-ksp_view` 2 flag at runtime.

```
KSP Object: 8 MPI processes
type: bcgs
maximum iterations=10000, initial guess is zero
tolerances: relative=1e-05, absolute=1e-50, divergence=10000.
left preconditioning
using PRECONDITIONED norm type for convergence test
PC Object: 8 MPI processes
type: bjacobi
number of blocks = 8
Local solver information for first block is in the following KSP and PC objects on rank 0:
Use -ksp_view ::ascii_info_detail to display information for all blocks
KSP Object: (sub_) 1 MPI process
type: preonly
maximum iterations=10000, initial guess is zero
tolerances: relative=1e-05, absolute=1e-50, divergence=10000.
left preconditioning
using NONE norm type for convergence test
PC Object: (sub_) 1 MPI process
type: ilu
out-of-place factorization
0 levels of fill
```

Fig. 2: `-ksp_view` output

It is also possible to visualize the matrix sparsity pattern using `-mat_view draw`, and get information about matrix properties like an estimate of maximum and minimum eigenvalues using `-ksp_singularvalues`. Some of the available *ksp* solver methods and preconditioners are listed below:

TABLE I: *KSP* Methods

Algorithm	Associated Type in PETSc
Richardson	richardson
GMRES	gmres
Flexible GMRES	fgmres
Conjugate Gradient	cg
Bi-Conjugate Gradient	bicg
Stabilized Bi-Conjugate Gradient	bcgs
Pipelined Conjugate Gradient	pipecg

TABLE II: *PC* Types

Algorithm	Associated Type in PETSc
Additive Schwarz	asm
Block Jacobi	bjacobi
algebraic multigrid	gamg
geometric multigrid	mg
incomplete LU	ilu
Gauss elimination(LU)	lu

A more comprehensive list of solvers and preconditioners is available here [3]. Information about residuals and iterations can be obtained using `-ksp_monitor` and the criteria for which convergence occurred can be checked with `-ksp_converged_reason`

B. *Grid Management*

PETSc provides another class *DM*(Domain Management) for managing grid connectivity and distribution in case of parallel processing. A *DM* object can be attached

to a solver(*KSP*, *SNES*) object via *KSPSetDM(KSP ksp, DM dm)*. It enables the solver to use the *DM* information to create and process data that the solver needs to set up. For instance, with `PCMG`, providing a *DM* helps it to create geometric multigrid on your problem. *DMDALocalInfo* is a C structure that contains information about global and local grid extents, which are also useful in some situations.

Currently, *DM* supports PDEs on structured and staggered grids with finite difference methods(*DMDA* and *DMSTAG*), PDEs on unstructured grids with finite element and finite volume methods(*DMPLEX*), PDEs on quad and octree-grids(*DMFOREST*), models on networks such as power grid or river networks(*DMNETWORK*), and particle-in-cell simulations(*DMSWARM*) [4].

C. Time Integrators

PETSc offers a variety of time integration methods, including explicit methods such as forward Euler and Runge-Kutta methods and implicit methods such as backward Euler and Crank-Nicholson. The *TS*(Time Stepping) object serves as the primary interface for configuring and controlling the time-stepping process. It allows setting the time integration method, time step size, and other parameters related to the time integration scheme. PETSc also offers various mechanisms for time step control. Automatic time step adaptation can be performed based on error estimates, stability conditions, or user-defined criteria. It also provides an error monitoring infrastructure for estimating the error in the solution and adjusting the time step size accordingly. Some of the time integrators are listed below [3]:

TABLE III: Time Integration Schemes

TS Name	Class	Type	Order
euler	one-step	explicit	1
ssp	Runge-Kutta	explicit	≤ 4
rk	Runge-Kutta	explicit	≥ 1

D. Non-Linear Solvers

The SNES library of PETSc offers a robust set of data-structure-neutral numerical procedures for nonlinear situations. The interface is the same for both sequential and parallel scenarios, with the difference being that in parallel cases, each process usually only contributes locally to the matrices and vectors. The user only needs to supply a routine to evaluate the nonlinear function to use the SNES solver. To set up this function, PETSc offers a customizable interface that enables you to specify the equation(s) and calculate the residual at each iteration. Figure 3 depicts the common structure of a

SNES application. The users must provide the functions inside the *user code* block. Also, the user directly acts on all the objects referenced by solid arrows, like the *DMDA* to discretize the problem on a grid(structured in this example), *Mat* for storing the Jacobian and *Vec* for storing the solution vector and residuals, and set the required *SNES* solver details. The *SNES* solver, in turn, already has *KSP* and *PC* objects, even though not explicitly asked for by the user, as they are required for the inner iterations. These can be set using the `-ksp_type` and `-pc_type`

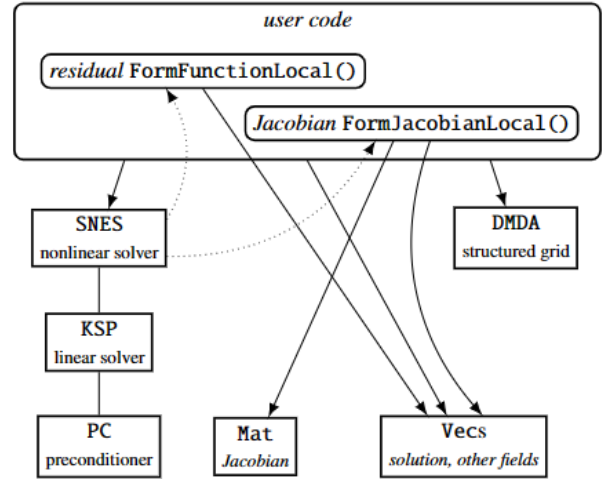


Fig. 3: SNES Structure Overview [6]

The *residual FormFunctionLocal* is the callback function for calculating the residual of the nonlinear system, while *Jacobian FormJacobianLocal* is the callback function for calculating the analytical Jacobian. Also, the user has the option to either provide analytical Jacobian or use one of the options from the following.:

- Finite Difference approximation of Jacobian using `-snes_fd`
- Finite Difference approximation with coloring using `-snes_fd_color`
- Just calculate the action of the Jacobian $J_F(x_k)y$ in a Krylov iterative method using `-snes_mf`

Some of the nonlinear solvers are listed below [3]:

TABLE IV: Nonlinear Solvers

Algorithm	Associated Type
Newton's Method	newtonls
Nonlinear GMRES	ngmres
Nonlinear Richardson	nrichardson

Nonlinear preconditioning in *PETSc* involves the use of an inner *SNES* instance to define the step for an

outer *SNES* instance. The inner instance can be set using `-npc_` prefix. Nonlinear preconditioning can be either of type left or right. The side may be selected using `-snes_npc_side`. Also, basic solver compositions are available with *SNESCOMPOSITE*. It gives users the option to combine two or more *SNES* instances additively or multiplicatively [5]. Using the command line, a set of *SNES* types can be given by a comma-separated list to `-snes_composite_sneses`

E. PETSc Profiling

In *PETSc*, there are often operations with big datasets like sparse matrices and the arithmetic intensity, i.e. *FLOPS/byte* ratio, is usually below unity. Also, most operations are curtailed by the memory bandwidth of the machine one runs on. It means that the memory bandwidth limits the maximum speedup you can obtain from an application, and after a certain threshold, increasing the number of cores will not necessarily lead to faster processing times. [8]

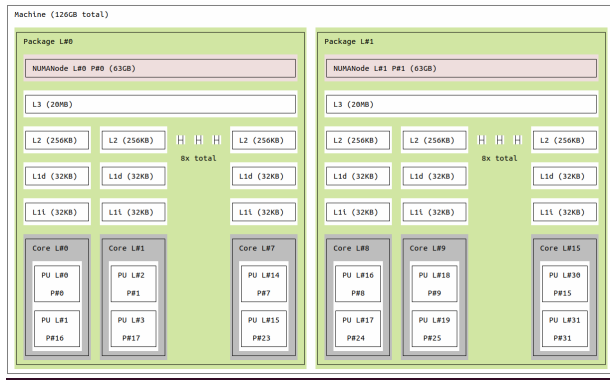


Fig. 4: Istopo output on SCCS login node

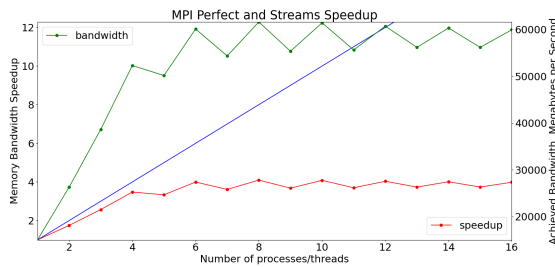


Fig. 5: Streams Benchmark

The above figure 5 shows the *streams* benchmark available in *PETSc*, on *sccs-gpu* cluster login node 5. One can run the *streams* benchmark by going into *PETSC_DIR* and running `make streams`. The *streams* benchmark calculates the streams triad $w = y + \alpha x$

for large sizes which exceed the cache size. It can be seen that naively running the *streams* benchmark, the speedup (red curve) saturates at around 4. Better results for the *stream triad* benchmark can be obtained by mapping the *mpi* processes by socket rather than cores. It was seen that `-map-by socket` is almost twice as fast as `-map-by core` for six processes. It shows that the selection of optimal mapping depends on the application in question and the software and hardware stack, and it is important to try different bindings before commenting on speedup.

PETSc also supports various profiling options to obtain information about the floating point operations, *MPI* operations, time taken by various stages of the application etc. Such information can be obtained by passing `-log_view` option at command line.

```

----- PETSc Performance Summary: -----
./fex19 on a linux-c-opts named x-wing0 with 1 processor, by ge26cqe Sun Jun  4 09:31:56 2023
Using Petsc Release Version 3.19.1, unknown

Time (sec):      2.824e+00      1.000      2.824e+00      Total
Objects:         8.800e+01      1.000      8.800e+01
Flops:           7.887e+09      1.000      7.887e+09      7.887e+09
Flops/sec:       2.792e+09      1.000      2.792e+09      2.792e+09
MPI Msg Count:    0.000e+00      0.000      0.000e+00      0.000e+00
MPI Msg Len (bytes): 0.000e+00      0.000      0.000e+00      0.000e+00
MPI Reductions:  0.000e+00      0.000      0.000e+00      0.000e+00

Flop counting convention: 1 flop = 1 real number operation of type (multiply/divide/add/subtract)
e.g., VecAXPY() for real vectors of length N --> 2N flops
and VecAXPY() for complex vectors of length N --> 8N flops

```

Fig. 6: Log view FLOPS

More details about the timings for various sections like KSP solve, SNES solve, PC application, Matrix multiplication etc. can be found inside `-log_view` as well.

```

Event      Count      Time (sec)      Flop      Flop-Byte-Ratio      Message-Bytes      Message-Bytes-per-Second      Global-Size      Local-Size      Stage      Total      GPU      GPU-ops      GPU-ops-per-Second      GPU-ops-per-Second
... Event Stage 0: Main Stage
MatVec      7      1.0      0.7887e+09      1.0      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00
MatVec      58      1.0      0.7887e+09      1.0      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00
MatVec      58      1.0      0.7887e+09      1.0      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00
MatVec      58      1.0      0.7887e+09      1.0      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00      0.000e+00

```

Fig. 7: Log view Operations

Events to log can be explicitly added to an application through *PetscLogEventBegin()*, *PetscLogEventEnd()* and also FLOPS can be registered by the user using *PetscLogFlops()*.

F. GPU support

PETSc algebraic solvers run on GPU systems from NVIDIA using CUDA and AMD and Intel using OpenCL/ViennaCL and HIP. Effective GPU implementations of low-level linear algebra operations provide a highly performant alternative solution strategy for users. *PETSc* follows a 'lazy-mirror' model. It tracks all *Vec* (and *Mat*) objects with the ability also to track "offloaded" copies of the data and synchronize these two copies of the data (only) when required [3].

TABLE V: GPU Support

Prog. Model	Package	Vec	Mat	GPUs
CUDA	cuBLAS,cuSparse	✓	✓	NVIDIA
HIP	Rocm	✓	✓	AMD
SYCL	MKL	X	X	-
OpenCL	ViennaCL	✓	✓	NVIDIA AMD Intel
Kokkos	Kokkos kernels	✓	✓	NVIDIA AMD Intel

G. Supported External Softwares

In addition to its extensive built-in capabilities, PETSc also offers support for external software integration, enabling users to leverage the functionality of other software tools and libraries. Some of them are listed below, and more details can be found here [3]

- Trilinos
- SLEPc (Scalable Library for Eigenvalue Problem Computations)
- SuperLU
- MUMPS (Multifrontal Massively Parallel Solver)
- Hypr (Algebraic multigrid solver)
- MOOSE: MOOSE (Multiphysics Object-Oriented Simulation Environment)

IV. CASE STUDIES

The following case studies were performed on the compute node of SCCS cluster. It is equipped with:

- 1x AMD EPYC 7402 @ 2.80GHz single socket, 24 cores per socket
- 2-way hyperthreading
- 256GB RAM
- 4x RTX 3080 Turbo with 10GB video RAM per each
- 512GB NVMe PCIe Gen4 SSD

A. Experiments with Nonlinear driven cavity in 2d

This example was taken from the tutorials of PETSc for SNES solvers. It has a 2D domain with a moving lid and also solves for temperature field. It uses velocity-vorticity formulation on a structured distributed grid parallelized with DMDA, with finite difference discretization. The system of PDEs reads as follows:

$$-\Delta U - \partial_y \Omega = 0 \quad (11)$$

$$-\Delta V + \partial_x \Omega = 0 \quad (12)$$

$$-\Delta \Omega + \nabla \cdot ([U * \Omega, V * \Omega]) - GR * \partial_x T = 0 \quad (13)$$

$$-\Delta T + PR * \nabla \cdot ([U * T, V * T]) = 0 \quad (14)$$

where U and V are the velocity fields,

Ω is the vorticity,

T is the temperature,

GR is the Grashof number,

PR is the Prandtl number

1) **Exact Vs Inexact linear system solver:** The default grid size is 16x16 and the linear system solver is GMRES with preconditioner ILU.

```
lid velocity = 100., prandtl # = 1., grashof # = 100.
0 SNES Function norm 7.681163231938e+02
Linear solve converged due to CONVERGED_RTOL iterations 79
1 SNES Function norm 6.582880149343e+02
Linear solve converged due to CONVERGED_RTOL iterations 81
2 SNES Function norm 5.294044895985e+02
Linear solve converged due to CONVERGED_RTOL iterations 89
3 SNES Function norm 3.775102017644e+02
Linear solve converged due to CONVERGED_RTOL iterations 103
4 SNES Function norm 3.047226675957e+02
Linear solve converged due to CONVERGED_RTOL iterations 145
5 SNES Function norm 2.599983493586e+00
Linear solve converged due to CONVERGED_RTOL iterations 198
6 SNES Function norm 9.427316093763e-03
Linear solve converged due to CONVERGED_RTOL iterations 140
7 SNES Function norm 5.198422344878e-08
Nonlinear solve converged due to CONVERGED_FNORM_RELATIVE iterations 7
Number of SNES iterations = 7
```

Fig. 8: Initial Run with default parameters

It can be seen that it takes 7 Newton's iterations, with the linear system solver taking between 80-150 iterations. We can change the preconditioner to `-pc_type lu` so that the linear solver is exact. On comparing the total time required till convergence for preconditioning with exact(LU) vs inexact(ILU) we get:

TABLE VI: Exact Vs Inexact Newton's iteration

PC	Time(s)
ILU(inexact)	0.3292
LU(exact)	0.7508
SUPERLU(exact)	3.87

It can see that time required for exact solver is more. So it might be better to use an iterative solver, which may take more iterations but result in a faster solver. We can try reducing the convergence tolerance of the "inner" KSP solves to see if it performs faster with the outer iterations still converging.

TABLE VII: Varying convergence tolerance

RTOL	Time(s)
1e-8	1.675
1e-5	0.8758
1e-3	0.6341
1e-2	0.5509
1e-1	diverged

It can be seen in table VII that the least time is obtained with a tolerance of $1e-2$ for the inner *KSP* solver, whereas for $1e-1$, it diverges as the line search of the outer *SNES* iteration fails for such low inner tolerance.

2) **Scaling the problem:** Newton's method is known to show quadratic convergence and is independent of the problem(grid) size [6]. However, it may not be the case for the inner *KSP* solver and is dependent on the preconditioner. To test this, we can increase the problem size by refining the grid gradually and see the effect on the linear solver iterations and the total time.

The default solver is *GMRES(30)* which fails for `-da_refine 4` case when running in parallel. Hence, for this section `-ksp_type` is set to *BCGS* and the preconditioner to *BJACOBI* with *ILU(0)* for each block. The number of *MPI* ranks are also kept constant at 8.

TABLE VIII: Comparison ILU vs MG

refine	SNES iter	size(nnz)	KSP iter		Time(s)	
			ILU	MG	ILU	MG
2	7	293776	60-100	5-6	0.175	0.161
3	6	1163536	90-240	5-8	0.78	0.374
4	5	4631056	160-500	6-7	7.48	1.414

It is evident from the table VIII that Newton's Solver shows mesh independence, i.e. the number of iterations for `SNES` solves takes roughly six iterations every time. However, the iterations for *ksp* solver increases with an increase in grid points with the preconditioner *ILU*. This leads to a substantial increase in the total time and hence is not scalable. On the other hand, the more powerful multigrid preconditioner *mg* is much better in terms of scalability, with the number of *KSP* iterations remaining almost constant. It also leads to lower total time, as is seen in table VIII.

3) **Non-linearity:** Another experiment that can be done to test the limits of nonlinear solver *SNES* is to increase the nonlinearity. In previous sections, the grashof number was kept constant at 100. In this section, it is raised to test the capabilities of *SNES* solver.

TABLE IX: Varying Grashof number refine_factor(2)

Grashof number	SNES iter	KSP iter
100	7	80-200
1000	7	80-265
10000	7	200-500
13000	diverged	diverged

It can be seen that the default linear solver *GMRES* fails at grashof number 13000 as it could not achieve the desired convergence tolerance within the maximum iterations(10000) specified. Changing the preconditioner to *PCMG* works for grashof number 13000. However, on trying to increase it more, it fails at GR13373. We could try changing to the exact solver using *PCLU*, but that does not help either.

4) **Composable Non-linear solvers:** Since Newton's method does not go beyond grashof number 13373 in this particular example, one option is to use nonlinear preconditioning [5]. First, we can try nonlinear Richardson iteration with Newton's method as the preconditioner. This can be done by setting `-snes_type richardson` `-npc_snes_type newtonls` `-npc_snes_max_it 4`. While this combination does not work with `-npc_pc_type mg`, it does with an exact preconditioner *LU*. But increasing the grashof number further to 14000 leads to failure of this combination too.

Another option is to use Newton's method preconditioned with nonlinear Richardson. Accordingly, setting `-npc_snes_type nrichardson` it does work X. The combination can be made even faster by increasing the maximum preconditioning iterations for the inner nonlinear Richardson solver X.

TABLE X: Varying - npc_snes_max_it

NPC max iterations	SNES iter	Time(s)
1	359	23.77
3	23	1.528
7	8	0.542

This combination of Newton's method, preconditioned with nonlinear Richardson can be pushed to grashof number 1000000. It shows highly nonlinear problems can be solved with nonlinear preconditioning.

5) **GPU vs CPU:** For comparison of CPU vs GPU, the following options were used `-da_refine 9 -pc_type mg -mg_levels_pc_type jacobi` for CPU and `-dm_vec_type` as *cuda* and `-dm_mat_type` as *aijcusparse*, for GPU. Timings

were taken by increasing the number of processes keeping the same problem size. For the GPU case, the number of GPUs correspond to the number of processes.

TABLE XI: CPU vs GPU comparison

Number of procs	Time(CPU)	Time(GPU)
1	87.51	25.86
2	54.75	18.31
3	32.96	13.4
4	32.4	11.11

TABLE XII: Comparison CPU vs GPU of multigrid-components

MG Level	Time(CPU)	Time(GPU)
Level 0	0.00116	0.014055
Level 1	0.00144	0.027273
Level 2	0.00197	0.029665
Level 3	0.00416	0.030666
Level 4	0.01227	0.030616
Level 5	0.04133	0.031719
Level 6	0.12351	0.03417
Level 7	0.50941	0.046373
Level 8	2.3629	0.11479
Level 9	9.9467	0.32745

It can be seen from table XI that GPU performs faster overall. The second table XII shows the timing comparison of multigrid levels. It can be observed for CPU that the time required for multigrid smoothing reduces as we go to coarser levels. However, though GPU is faster overall, the coarser levels take longer to solve and are not necessarily faster than the finer levels beyond a certain point. In fact, the levels 0-4 are faster on CPU than on GPU. It might be due to the latencies involved in kernel launches on GPU, and this time may dominate for coarser levels. This becomes especially important when running multigrid with W cycles, which visit coarser levels multiple times.

B. Experiments with Turbulent flow on HPC Systems lab code (NS-EOF)

The code for this case study was taken from the course *Turbulent Flow on HPC Systems(IN2311)* taught at TUM. The code framework is named NS-EOF. It can be used to simulate two- and three-dimensional incompressible Navier-Stokes problems on Cartesian grids. The solver uses finite difference discretization. It uses explicit Euler time-stepping for the evolution of the flow velocities and an implicit formulation for the pressure. The resulting Pressure-Poisson equation is solved using the PETSc library. The assembly of the

matrix and right-hand side as per ranks is done inside a separate class *PETScSolver*. The default solver used for solving the linear system is *KSPFGMRES restart(30)* with preconditioner *PCILU* for serial case and *PCASM* for parallel case. With these experiments, an attempt was made to modify the existing solvers and check whether they performed any better than the default configuration. For experiments, two cases were selected.:

- 1) *Cavity2D* which is a standard lid-driven cavity with laminar flow, *Reynold's number* = 500 on a 100x100 grid
- 2) *Channel2DTurbulent* which is a *Pouseille flow* problem including turbulence which is modelled using *Algebraic Turbulence* model, with 25x40 grid and *Reynold's number* = 10000

TABLE XIII: Cavity 2D 100x100 on 4 processors

KSP	PC	SUB-PC	Time	Speedup
FGMRES	ASM	ILU	18.019	1 (default)
FGMRES	BJACOBI	ILU	20.376	0.884
FGMRES	ASM (overlap2)	ILU	17.843	1.01
BCGS	ASM	ILU	10.831	1.664
BCGS	ASM (overlap2)	ILU	11.164	1.614

The experiments were done on SCCS cluster compute node. It can be seen from table XIII that *BCGS* performs slightly better than the default solver *FGMRES*, for the Cavity2D case. However, in table XIV, it can be seen that *FGMRES* is faster than *BCGS*. It might be because while *BCGS* is more efficient and faster in the presence of a "good" preconditioner, *FGMRES* is more robust and works on a wider range of matrices [7].

TABLE XIV: Channel 2D Turbulent 25x40 on 4 processors

KSP	PC	SUB-PC	Time	Speedup
FGMRES	ASM	ILU	16.580	1 (default)
FGMRES	BJACOBI	ILU	15.981	1.0374
FGMRES	ASM (overlap2)	ILU	16.721	0.992
BCGS	ASM	ILU	20.0814	0.826
BCGS	ASM (overlap2)	ILU	25.394	0.653

TABLE XV: CPU Vs GPU

grid size	CPU time	GPU time
25x25	0.63	8.195
50x50	6.01	39.999
75x75	22.384	107.577
100x100	57.041	214.249

The above table XV compares CPU, and GPU runs for Channel 2D turbulent case. It can be seen that GPU performs worse than CPU for all the configurations. Whether speedup will be obtained by running on GPU largely depends on the underlying algorithm used and whether it benefits from the kind of parallelism GPU offers. Most non-trivial PETSc operations do not benefit from the modern GPUs because of the composability of GPU codes [8].

V. CONCLUSION AND FUTURE WORK

In conclusion, this paper has presented a comprehensive overview of the PETSc (Portable, Extensible Toolkit for Scientific Computation) library, highlighting its capabilities and demonstrating its effectiveness through two case studies. The first case study focused on applying PETSc in solving a nonlinear driven cavity problem. The second case study explored its usage in simulating turbulent flow on HPC systems. Its nonlinear solver's *SNES* capabilities allowed for efficient, accurate solutions and extensive experimentation with the nonlinear driven cavity problem. The flexibility of PETSc's linear solver enables users to select the most suitable solver and architecture (CPU or GPU). It was also seen that not all operations from PETSc benefit from the GPU architecture, and it is upto the user to check its applicability and make an informed decision.

While PETSc undoubtedly offers a lot of flexibility and choice to its users, it is important to note that the learning curve for PETSc may be steep just because of the sheer size of the library and its applications. It also has many configuration options, and selecting the optimum one is a task in itself. However, PETSc has comprehensive documentation [3] and active community support, which can help with these problems.

VI. ACKNOWLEDGEMENTS

I would like to extend my heartfelt gratitude to my seminar advisor, Mr. Hayden Liu Weng for his invaluable guidance, patience and support throughout seminar. I would also like to express my gratitude to SCCS for providing me access to cluster to conduct my studies. Also, thanks to Mr. Fabio Gratl and Mr. Markus Mühlhäußer for their prompt support with the organizational aspects of the seminar.

REFERENCES

- [1] Petsc code. <https://gitlab.com/petsc/petsc>, 2013.
- [2] Anzt, Hartwig, Cojean, et al. Ginkgo: A Modern Linear Operator Algebra Framework for High Performance Computing. *ACM Transactions on Mathematical Software*, 48(1):2:1–2:33, February 2022.
- [3] Satish Balay, Shrirang Abhyankar, Mark F. Adams, et al. PETSc Web page. <https://petsc.org/>, 2023.
- [4] Satish Balay, Shrirang Abhyankar, Mark F. Adams, et al. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.19, Argonne National Laboratory, 2023.

- [5] Peter R. Brune, Matthew G. Knepley, Barry F. Smith, and Xuemin Tu. Composing scalable nonlinear algebraic solvers. *SIAM Review*, 57(4):535–565, jan 2015.
- [6] Ed Bueler. *PETSc for Partial Differential Equations: Numerical Solutions in C and Python*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2020.
- [7] Aditi Ghai, Cao Lu, and Xiangmin Jiao. A comparison of preconditioned krylov subspace methods for large-scale nonsymmetric linear systems, 2018.
- [8] Richard Tran Mills. Understanding and tuning performance in petsc (on emerging manycore, gpgpu, and traditional architectures). <https://www.mcs.anl.gov/petsc/meetings/2019/slides/mills-performance-gpus-tutorial-petsc2019.pdf>, 2019. Accessed: 2019–06-04.
- [9] Richard Tran Mills. Introduction to non-linear solvers using petsc/tao. <https://xsdk-project.github.io/MathPackagesTraining2020/files/ATPESC-2020-Track-5-Talk-7-Mills-NonlinearSolversPETSc.pdf>, 2022. Accessed: 2022–08-09.